RetroNNetScaler: Hard-Palette Realtime Upscaling for Classic NES Pixel Art

A systems and learning report on integer Clarity, sharp xBR geometry, and discrete neighbor-classification CNNs for 4K presentation

RetroNNetScaler Project

Technical Report | July 2026 | https://github.com/amirsdream/RetroNNetScaler

Abstract

Abstract. Upscaling palette-limited pixel art for modern 4K displays is fundamentally different from natural-image super-resolution. Methods that minimise continuous reconstruction error (bilinear, ESRGAN-class networks) introduce mid-tone cream colours and silky outlines that destroy the aliased aesthetic of 8-bit games. We present **RetroNNetScaler**, a Windows realtime pipeline that embeds a NES core (SimpleNES), presents frames through Direct3D 11, and offers three production paths: *Clarity* (maximum integer nearest), *Smooth* (Hyllian-style xBR-sharp with hard neighbourhood snapping), and *Neural* (a tiny 9-class neighbour classifier trained against Scale2xSFX teachers). We show that residual RGB regression underfits sparse diagonal structure and that unconstrained discrete classifiers create multicolour edge fringes when false-positive neighbour picks exceed about 1% of pixels. A tip-gated, hard-palette design - and falling back Neural presentation to sharp xBR when the classifier is unreliable - preserves discrete NES colours while improving perceived diagonals. The network (about 52k weights, C=32) is small enough for future FPGA mapping. We report training setup, validation metrics, failure modes, and an FPGA-oriented roadmap.

Keywords: pixel-art upscaling, NES, xBR, Scale2xSFX, discrete super-resolution, realtime D3D11, FPGA-oriented CNN

1. Introduction

Classic console graphics were authored on a square pixel lattice with a small discrete palette. When those frames are shown on a 3840x2160 panel, naive bilinear or bicubic filters blur edges and invent colours that never existed in the ROM. Recent deep super-resolution models, trained for photographs or anime with continuous losses, tend to produce a *silky* look: soft outlines, chroma bleed, and palette drift. For retro enthusiasts and preservationists, that look is a regression.

Rule-based pixel-art scalers such as EPX/Scale2x, Scale2xSFX, HQX, and xBR/xBRZ address diagonals by copying existing neighbour colours rather than interpolating. Kopf and Lischinski depixelization goes further toward resolution-independent vectors, which many players perceive as too smooth for classic NES draw. Our product goal is different: **hard colours, cleaner 45-degree diagonals, realtime 4K presentation**, with a path toward FPGA implementation.

This report documents the RetroNNetScaler prototype: system architecture, shader algorithms, a CUDA training pipeline for a discrete TinySR classifier, quantitative validation on Super Mario Bros. dumps, and lessons from failure modes (cream residuals, edge colour explosions).

2. Related Work

2.1 Classical pixel-art scalers

EPX/Scale2x expands each pixel into a 2x2 block using equality tests on the 3x3 neighbourhood, never inventing colours. Scale2xSFX (and Scale3xSFX) refine diagonal decisions with a wider support and reduce hollow-diamond artifacts. xBR and xBRZ detect edges with weighted colour distances (often in YUV) and write tip wedges; Hyllian-style sharp variants avoid blending. These methods remain the gold standard for hard-palette look.

2.2 Depixelization and continuous SR

Kopf-Lischinski depixelization extracts smooth contours suitable for vector magnification. Deep unsupervised pixelization and modern GAN/transformer SR (Real-ESRGAN, SwinIR, anime SR) optimise perceptual or continuous losses. Structure-aware pixel-art work (2026) emphasises that continuous models violate discrete palette integrity. Quantized generators (for example Gumbel-softmax palette models) align better with our constraint of no new colours.

2.3 Tiny learned upscalers

ESPCN-style networks with pixel-shuffle heads are attractive for realtime and FPGA targets because of small parameter counts. We adopt that footprint but replace RGB residual regression with a **9-way neighbour classifier**, so every output texel is exactly one of the nine colours in the local LQ neighbourhood.

3. System Architecture

The PC prototype is organised as a single process:

```
NES core (SimpleNES)
     |  256x240 RGBA @ ~60.098 Hz
     v
D3D11 upload  ->  scaler compute shader  ->  letterbox present
     |
Modes: Nearest | Bilinear | NIS | Clarity | Smooth | Neural
```

A custom Win32 launcher provides ROM selection (browse / drag-drop), scale mode chips (Clarity / Smooth / Neural), and output resolution (1080p / 1440p / 4K). Preferences persist under AppData. Gameplay is paced to the NES NTSC rate independent of panel refresh (for example 165 Hz).

Table 1. Scaling modes in the current build.

Mode	Algorithm class	Palette behaviour
Clarity	Max integer nearest	Exact LQ colours
Smooth	xBR-sharp tips + snapExact9	Hard neighbourhood only
Neural*	TinySR 9-class (safe: xBR)	Hard; CNN gated when unsafe
NIS	NVIDIA Image Scaling	Continuous (reference)
Nearest / Bilinear	Baselines	Exact / blended

**Neural presentation uses sharp xBR when the classifier would fringe; CNN weights remain in-tree for research.*

4. Methods

4.1 Clarity - integer oracle

Clarity selects the largest integer scale factor that fits the window, point-samples NES texels into that canvas, and letterboxes. This is the reference Pure look: blocky but colour-perfect. Any subsequent filter must beat Clarity on diagonals without introducing cream.

4.2 Smooth - sharp xBR with hard snap

Smooth implements a Hyllian-inspired xBR-sharp pass in HLSL. For each output sample we evaluate edge dominance with a YUV colour distance, restrict writes to outer tip wedges of each quadrant, apply an outline guard on high-contrast corners, and finally snapExact9: the result is replaced by the nearest of the nine LQ neighbourhood colours. Mid-tones cannot survive the snap, which eliminates cream while keeping intentional tip geometry.

4.3 Neural - discrete neighbour classification

We train a tiny convolutional network that, for each LQ pixel, predicts a class in {A,B,C,D,E,F,G,H,I} for every subpixel of a 4x4 block (pixel-shuffle head). The decoded colour is exactly neighbour[argmax]. Topology:

```
Conv 3x3, C=32, ReLU  ->  Conv 3x3, C=32, ReLU  ->  Conv 3x3, 9*4*4
-> pixel_shuffle(r=4)  ->  logits (9, H*4, W*4)
Parameters ~51.8k  |  Dropout2d during train  |  AdamW + wd
```

Labels come from a Scale2xSFX teacher chained to 4x; each HQ pixel is mapped to the nearest of the nine LQ neighbours (exact match preferred, YUV distance fallback). Non-centre classes are rare (about 0.3-1% of pixels), so training uses class-weighted cross-entropy with a large edge multiplier (for example 32x) and early stopping on edge-class accuracy.

4.4 Why residual RGB failed

An earlier residual ESPCN (nearest + g*tanh(res)) matched teacher L1 closely after regularisation, but with g <= 0.55 could not reach full neighbour swaps. Raising gain without discrete decoding produced soft fringes. Critically, EPX/SFX deltas occupy much less than 1% of HQ pixels; unmasked L1 is dominated by flat regions, so the net learns near-zero residual. Masked residual loss improved corner agreement (about 88% after hard snap) but stroke statistics still lagged a pure geometric teacher.

5. Training Pipeline

Frames are dumped from the live emulator (--dump-frames N), converted by build_dataset.py into LQ/HQ PNG pairs plus uint8 neighbour-index maps, and trained with train_tinysr.py on NVIDIA CUDA (validated on GeForce RTX 4080, PyTorch 2.11 + cu128). Export writes tinysr_weights.bin (magic TNSR, channels, scale, then W1 B1 W2 B2 W3 B3 floats) for StructuredBuffer load in the compute shader.

Table 2. Classifier training configuration (final run).

Item	Value
Teacher	Scale2xSFX x2 -> 4x
Dataset	~240 train / 60 val SMB frames (stride 1)
Crop / aug	96x96 LQ, rot90 + flips, edge-biased crops
Optimiser	AdamW, lr 1.5e-3 cosine, wd 2e-3
Regularisation	Dropout2d 0.1, grad clip 1.0, patience 20
Batch / device	48, CUDA GPU (no CPU offload)
Export	C=32, scale=4, 9 classes, 51760 floats

6. Experiments and Results

We evaluate on held-out SMB validation frames. Metrics: overall and edge-only neighbour accuracy (edge = label != E), L1 vs teacher, edge-map L1, palette drift (fraction of HQ pixels outside the 3x3 LQ set), and a stroke-variance proxy on strong edges.

Table 3. Validation means (12-frame subset, classifier checkpoint).

Metric	Nearest	Neural CNN	Teacher (SFX)
Neighbour acc (all)	-	96.4%	100% (by def.)
Neighbour acc (edge)	-	71.8%	100%
L1 vs teacher	0.00279	0.00810	0
Edge-map L1 vs teacher	0.0206	0.0559	0
Palette drift	0	0	0
Stroke variance proxy	0.892	0.939	1.231

Palette drift is identically zero for the discrete decoder - an essential product constraint. Edge accuracy of about 72% is meaningful but insufficient: the remaining errors concentrate on glyph and sprite silhouettes and are visually catastrophic (Section 7). Stroke variance of the CNN lies between nearest and teacher, indicating partial geometric learning.

7. Failure Analysis

7.1 Silky / cream look

Continuous residuals and any path that averages colours across an edge create mid-tones. Mitigation: hard snap to the LQ neighbourhood; forbid perceptual / adversarial losses aimed at natural images.

7.2 Exploded colours on edges

When the classifier assigns a false non-E class, the shader copies a neighbour that may be a completely different palette entry (for example red HUD vs black background). At 4x these errors appear as multicolour static around letters and sprites. Weight layout verification against PyTorch confirmed correct indexing; the fault is statistical, not binding. Mitigations implemented: (i) strong logit margin vs centre E; (ii) tip-only acceptance masks per quadrant; (iii) runtime presentation fallback of Neural to sharp xBR until edge accuracy is reliable.

7.3 Sparse teacher signal

Only a fraction of a percent of HQ pixels differ from nearest under EPX/SFX. Unweighted losses drown the diagonal signal. Class-balanced CE and edge-biased crops help but do not yet match Smooth geometric consistency on text.

8. FPGA and Future Work

The classifier parameter count and 3x3 conv structure map naturally to streaming FPGA pipelines (line buffers + MAC arrays). Intended lab board: DE10-Nano for Clarity/TinySR; larger devices if transformer Regen is pursued on PC first. Next research steps:

- Tip-only label generation (supervise only xBR/SFX tip loci).
- Two-head model: edge vs flat binary gate + 8-way direction on tips.
- Multi-game dumps to reduce SMB-specific overfitting.
- Distill Smooth xBR decisions as an alternative teacher to SFX.
- On-device int8 export and cycle-accurate RTL for the stem convolutions.

9. Conclusion

RetroNNetScaler demonstrates a complete path from NES emulation to 4K presentation with explicit respect for discrete palettes. Sharp xBR with neighbourhood snap currently delivers the best realtime diagonals without silk. A FPGA-sized neighbour-classification CNN achieves zero palette drift and partial edge learning, but unconstrained argmax decoding fringes until tip gating and higher edge accuracy are achieved. The project central lesson is methodological: for pixel art, **discretise the output domain first**, then learn; continuous SR objectives fight the medium.

References

[1] A. Mazzoleni, Scale2x / Scale3x, scale2x.it - AdvMAME pixel-art scalers.

[2] Scale2xSFX / Scale3xSFX proposals (libretro community; ScaleNx implementations).

[3] H. Hyllian et al., xBR / xBRZ family - edge-directed pixel-art upscaling.

[4] J. Kopf and D. Lischinski, Depixelizing Pixel Art, ACM TOG (SIGGRAPH), 2011.

[5] W. Shi et al., Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network (ESPCN), CVPR, 2016.

[6] Structure-Aware Pixel Art Scaling via Block Size Detection, Appl. Sci., 2026.

[7] SD-piXL: Generating Low-Resolution Quantized Imagery via Score Distillation, 2024.

[8] SimpleNES - github.com/amhndu/SimpleNES (NES emulator core).

[9] NVIDIA Image Scaling (NIS) SDK - spatial upscaler reference.

[10] RetroNNetScaler source - github.com/amirsdream/RetroNNetScaler, 2026.

Correspondence via the project repository. ROM dumps used for training must be legally obtained by the user; ROMs are not redistributed.